

HPATR Codebase Audit

HPATR Codebase Audit

Date: 2026-03-31 Branch: Main (commit 5a3ed97) Auditor: Claude (automated)

A. Master File Audit Table

1. Imaging / Shadowgraphy

File Path	Inputs	Outputs	Prerequisites	
src/imaging/expansion_detection.py (1466 lines)	BGR image path or CLI arg; 13 tunable params: denoise_ksize, clahe_clip, clahe_tile, gamma, blur_ksize, canny_thresh1/2, hough_min/max_radius, hough_param1/2, hough_min_dist; optional output_dir	Dict with 13 keys: final_image (annotated RGB), optimized_circles, optimization_data (per-circle DataFrame-ready dicts), canny_circles, watershed_circles, circle_sources, all_circles_before_optimization, position_optimization_data, size_optimized_circles, watershed_debug_images, canny_edges, preprocess_debug, preprocess_params	opencv-python, numpy, matplotlib, pandas	5-step pipeline: (1) Merge → normalize → Gaussian → minEnclosingCircle px²); (3) Otsu threshold Watershed markers → Merge + deduplicate (2) Two-phase optimisation gradient descent to max size expansion/contract constants: MIN_DROPLET SMALL_DROPLET_MAX_A WATERSHED_DISTANCE MIN_CONNECTED_COMPONENTS reads from LaCie Phantom/Flashed_Out LaCie Shadowgraph/Expansion
src/imaging/save_and_analyse.py (550 lines)	Image path (TIFF from Phantom); expansion_detection.process_image() results	FINAL_OPTIMIZED_RESULT.png, FINAL_ANALYSIS.csv, CANNY_EDGES.png, circle visualisations, Hough visualisations — all saved to LaCie Shadowgraph/{timestamp}/	LaCie drive mounted; config_loader; expansion_detection	Orchestrates full CV pipeline subfolder. Uses get_image_has_todo to integrate
src/imaging/mp4_to_tiff.py (213 lines)	MP4 video path OR TIFF folder path; output folder	Individual TIFF frames (frame_XXXXXX.tiff); flashed_output.tiff (brightest frame)	opencv-python; config_loader	Two functions: video_to_tiff, find_brightest_frame grayscale intensity. Use
src/imaging/capture_still_image.py (114 lines)	Camera index; optional output path; optional resolution	Single PNG image saved to disk	USB webcam connected; opencv-python	Simple webcam capture CLI tool.
src/imaging/webcam_preview.py (278 lines)	Camera index	Live preview window; saved PNG snapshots on 's' keypress	USB webcam connected; opencv-python	OpenCV trackbar-based brightness/contrast/saturation Interactive GUI (not he

2. Spray Cone Analysis

File Path	Inputs	Outputs	Prerequisites	Notes
Trials/Cone_2.py (363 lines)	Image path (auto-finds Nicer_Spray.*); optional CSV params file	Cone_2_annotated.png, Cone_2_mask_overlay.png, Cone_2_mask.png, step images in cone_steps_2/	opencv-python, numpy; config_loader	Row-based cone detector. 18 tunable constants (CLAHE, gamma, adaptive threshold, morphology, line fitting). Can load params from CSV (same format as cone_2_annotated image).
Trials/Cone_3.py (1292 lines)	Image path (any cv2-readable format) via --input; auto-finds Cone_Trial_Image.* or Spray_1.* in script dir; optional --nozzle-x, --output, --no-open	{stem}_cone3{suffix} annotated BGR image; cone_3_debug/{stem}_debug.png (2x3 step grid) when DEBUG_MODE=True	opencv-python, numpy; config_loader (for imaging path defaults only)	PCA contour method. Pipeline: load → normalize to uint8 → Gaussian blur threshold → invert-if-needed → morphological close (k=7) → border cleanup → contour filter (strict/relaxed/fallback) → combine vertically-aligned contours (or centroid) → bin-by-y outermost points per edge → numpy.linalg.eigh eigendecomp → angle from dot product. Crops top 5% of image to remove background. detect_cone_angle() is the callable API, returns (angle_deg, annotated_image). Constants: MIN_CONTOUR_AREA_RATIO=0.01, MORPH_CLOSE_KSIZE=7. No scaling.
Trials/Cone_4.py (922 lines)	Image path (any cv2-readable) via --input; optional --nozzle-x, --no-open, --output	{stem}_cone4{suffix} annotated BGR image; cone_4_debug/{stem}_debug.png (6-panel matplotlib/OpenCV grid)	opencv-python, numpy, scipy (curve_fit); config_loader; matplotlib (optional, for debug scatter plots)	Sigmoid profile fitting — no thresholding or contours. Designed for diffusive (effervescent atomisation). Algorithm: crop top 17% → per-row: find darkest pixel → fit logistic sigmoid to left and right window (size 40% of half-row) using scipy.optimize.curve_fit; accept if R²≥0.75 and contrast≥10 → collect boundary x-coords → outermost bins → RANSAC line fit (x=m·y+c, 200 iterations, 15px inlier threshold) per row → normalised direction vectors. Returns fit_quality (mean line R²); warns if fit_quality<0.5. API: detect_cone_angle(image_path, nozzle_x=None, top_crop_ratio=0.17, debug=True, min_r2=0.75, min_contrast=10.0) → (angle_deg, annotated_image). Trials/Cone_Tester.py(604 lines) Image path; optional apex crop ratio interactive); optional-use-matplotlib Cone_Tester_annotated.png,Cone_Tester_mask_overlay.png,Cone_Tester_step_images opencv-python

File Path	Inputs	Outputs	Prerequisites	Notes
				matplotlib (fallback click);config_loader Hybrid approach: conv fitting with multiple fallbacks. Interactive apex selection via matplotlib. Trials/cone_GUI.py(1268 lines) Image path (auto-findsNice dir, or Load Image button); interactive slider tuning Live multi-panel display in GUI (no file output by default); saves annotated PNG; "Save/Load Params" reads/writes CSV tki (TkAgg, FigureCanvasTkAgg), opencv-python, numpy, PIL (Image, ImageTk);config_loader **Independent row-based cone detector with GUI** - NOT a wrapper around Cone_2. Ownprocess_image(): CLAHE - gamma correction -> piecewise weight mask (top/mid/bot thirds) -> adaptive mean threshold -> auto-invert -> morphological open+close -> row-scan top fraction of image for leftmost/rightmost fg pixels -> numpy.linalg.lstsq line fit per side -> angle from arctan of slopes. 15 (CLAHE_CLIP, CLAHE_TILE, BLUR_KSIZE, GAMMA, TOP_RAMP_SLOPE, WEIGHT_TOP/MID/BOT, ADAPT_BLOCK, ADAPT_C, FG_MIN_RATIO, CLOSE_K, TOP_BAND_FRAC, MIN_FG_PER_ROW, LINE_FIT_STAR, APEX_CLAMP_FRAC, LEFT_SLOPE_MAX, RIGHT_SLOPE_MIN). GUI scrollable controls panel; right 3-row image grid (original+scan-band overlay, gamma, weighted, mask, lines / previous annotated, latest annotated). ⓘ in matplotlib diagram popups per parameter. 200ms debounce on slider change

3. ML / Detectron2 / Inference

File Path	Inputs	Outputs	
src/ai/process_run.py (342 lines)	frames_folder (TIFF sequence from Phantom); output_folder; pixels_per_mm	flushed_output.tiff, ai_result.png (mask overlay), ai_results.json, ai_summary.xlsx	L { d m
AI/evaluate_model.py (891 lines)	Model .pth path; validation dir with images + annotations JSON; score threshold	Per-image Excel (evaluation_results.xlsx); visualisations + JSON per image in timestamped Validation_100 subfolder; summary.json; appends to evaluation_results.csv	d
AI/Training_Analysis/evaluate_model.py (787 lines)	Model .pth path (auto-detect via --model, or manual --config/--weights/--annotations/--images); optional --output, --score-threshold (default 0.5), --model-name, --no-vis, --validation-dir	Excel evaluation_results.xlsx (3 sheets: Model Summary, Per Image Results, Instance Details); summary.json; visualizations/ + json/ per-image outputs in Validation_100/{model_name}_{timestamp}/	d o
AI/Training_Analysis/train_detectron2.py (892 lines)	COCO annotations JSON + images dir (hardcoded D:\Experiments\TrainingData\Detectron_Trial_4\); resume model path	Trained model checkpoints; loss curve plots in plots/; metrics JSON; validation evaluation	d n
AI/Training_Analysis/inference_detectron2.py (406 lines)	Image paths or directory; model .pth path; optional px/mm calibration	visualizations/, masks/ (per-detection PNGs), data/ (per-image JSON), summary.json	d
AI/Training_Analysis/plot_from_metrics.py (100 lines)	metrics.json from Detectron2 training (hardcoded path: training_2025_12_26_10_03_18)	loss_curve.png in plots/ subfolder	n
AI/Training_Analysis/setup_training_dataset.py (105 lines)	Source dir (hardcoded blur_2025_12_23_20_21_35); dest dir (hardcoded Detectron_Trial_1)	Copied images + annotations.json in Detectron2 structure	s
AI/Training_Analysis/test_inference_debug.py (103 lines)	Model path (hardcoded training_2025_12_24_19_37_46); test images dir	Console output: detection counts at various thresholds	d

File Path	Inputs	Outputs	
AI/Training_Analysis/verify_installation.py (46 lines)	None	Console output: Python, PyTorch, CUDA, Detectron2 versions	te
AI/Training_Analysis/visualize_coco_results.py (380 lines)	coco_instances_results.json (hardcoded path: training_2025_12_25_15_43_57)	score_distribution.png, category_distribution.png, predictions_per_image.png, bbox_sizes.png, score_vs_category.png, summary_report.txt	n
AI/Training_Analysis/visualize_model_predictions.py (182 lines)	Model path (hardcoded training_2025_12_24_19_37_46); test images dir	Side-by-side comparison PNGs in visualization_debug/	d
AI/Training_Analysis/check_image_identity.py (121 lines)	Training images dir (Detectron_Trial_2); Validation_100 images dir	Console output: hash comparison results	h

4. Synthetic Data Generation

File Path	Inputs	Outputs
AI/synthetic_data/blur_spray_dataset_generator.py (2080 lines)	Module-level constants (all configurable): NUM_IMAGES=1000, IMAGE_WIDTH=1280, IMAGE_HEIGHT=800, CUSTOM_OUTPUT_FOLDER="Detectron_Trial_4", OUTPUT_DETECTRON_ONLY=True, NUM_WORKERS=None (auto), droplet/ligament/background params	{OUTPUT_BASE_DIR}/Detectron_Trial_4/image_x N + annotations.json (COCO format, RLE r visualizations/, sidebyside/ (suppressed by OUTPUT_DETECTRON_ONLY=True)
AI/synthetic_data/conjoined_blur_spray_dataset_generator.py (2221 lines)	Same module-level constants as blur generator, with key differences: NUM_IMAGES=10, CREATE_SIDE BYSIDE_VIS=True, OUTPUT_DETECTRON_ONLY=False, CUSTOM_OUTPUT_FOLDER=None (timestamped)	Same COCO format output as blur generator, wit also created
AI/synthetic_data/spray_dataset_generator.py (1175 lines)	Same as above	Same format output

5. Root-Level Duplicates

File Path	Inputs	Outputs	Prerequisites	Notes
inference_detectron2.py	Same as AI/Training_Analysis/inference_detectron2.py	Same	Same	Duplicate. MODEL_PATH points to Claudia.pth instead of training_2025_12_25_15_43_57. Otherwise identical structure.
setup_training_dataset.py	Same as AI/Training_Analysis/setup_training_dataset.py	Same	Same	Duplicate. Identical content.
train_detectron2.py (1289 lines)	Same as AI/Training_Analysis/train_detectron2.py	Same	Same + csv, PIL, tqdm	Evolved version. Has additional CSV logging, get_detection_dataset_dicts import, Visualizer import, wraps from functools. More features than AI/Training_Analysis/ copy.

6. GUI / Interface

File Path	Inputs	Outputs	Prerequisites	Notes
src/gui/GUI_Clean.py (2988 lines)	Serial port (Portenta H7); Phantom camera	Serial commands to Arduino; experiment logs (JSON/Excel) to LaCie; pressure/motor control;	PySide6, pyqtgraph, pyserial, numpy, pandas, matplotlib, PIL; optional: pyphantom,	Active redesign replacing Windows_Experiment_GUI. Apple-inspired dark theme, Fusion style. Left panel: shadowgraph display, live pressure chart, motor travel,

31/03/2026, 12:31		HPATR Codebase Audit		
File Path	Inputs	Outputs	Prerequisites	Notes
	(100.100.100.1); camera capture trigger; user interactions	camera capture trigger; cone analysis via Cone_4	pyvisa; config_loader; Cone_4.detect_cone_angle	emergency stop. Tabs: Hardware, Camera, Experiment, AFG (+ hidden How To). Imports Cone_4 for cone analysis.
src/gui/Windows_Experiment_GUI.py (3359 lines)	Serial port (Portenta H7); Phantom camera; user interactions	Same as GUI_Clean.py	customtkinter, pyserial, numpy, pandas, matplotlib (TkAgg), openpyxl, PIL; optional: pyphantom, pyvisa; config_loader	Legacy GUI. CustomTkinter dark theme. Full experiment workflow: motor homing, pressure control, camera capture, image analysis. TODOs in code for pressure controller, notes section.

7. Hardware Comms (Arduino / Firmware)

File Path	Inputs	Outputs	Prerequisites	Notes
src/gui/Pressure_Motor_Portenta.cpp (596 lines)	Serial commands from GUI: SPEED:X;DIST:Y, PRESSURE:X, PRESSURE_OFF, HOME:, RESET:, CAM:...	Serial responses: ARDUINO_READY, PRESSURE_SET:X, PRESSURE_READING:X.XX, PRESSURE_DISABLED, PROGRESS:X, MOVE_COMPLETE, CAM:... (relay)	Arduino Portenta H7; TMCStepper lib; AccelStepper lib; TMC5160 stepper driver; AliCat pressure controller (0-5V via logic level converter)	Firmware for combined motor + pressure control. TMC5160 stepper: 13600 steps/mm, 16 microsteps, 2A RMS current. Pressure: 12-bit PWM (20-68% duty = 0-26.4 BAR), 12-bit ADC read with calibration correction and rolling average (4 readings/1s). Optical endstops for homing. UART bridge (Serial1) for camera commands to Pi. Stall detection. Periodic 3s handshake.

8. Config / Utilities

File Path	Inputs	Outputs	Prerequisites	Notes
src/config_loader.py (275 lines)	config/paths.yaml or config/paths_example.yaml; OS environment	Config dict with resolved paths	PyYAML (optional — degrades gracefully); pathlib, platform	Auto-detects LaCie drive (macOS /Volumes/LaCie, Windows D:~Z: by folder presence, Linux /media). Resolves {lacie_drive} placeholders. Provides get_imaging_config(), get_gui_config(), get_mp4_config(). Global lazy-loaded singleton. win32api import for Windows volume label (optional).
src/__init__.py	—	—	—	Package marker (empty).
src/ai/__init__.py	—	—	—	Package marker (empty).
src/gui/__init__.py	—	—	—	Package marker (empty).
src/imaging/__init__.py	—	—	—	Package marker (empty).

B. Master Dependencies List

Python pip Packages

Package	Version Constraint	Listed in requirements.txt?	Used By
opencv-python	>=4.5.0	Yes	Nearly all files
numpy	>=1.20.0,<2.0	Yes (NumPy 2.x breaks pyphantom)	Nearly all files
matplotlib	>=3.3.0	Yes	Plotting, cone GUI, generators, save_and_analyse
pandas	>=1.2.0	Yes	Evaluation, GUI, save_and_analyse
customtkinter	>=5.0.0	Yes	Windows_Experiment_GUI.py only
Pillow	>=8.0.0	Yes	GUIs (Image/ImageTk), cone_GUI, train_detectron2
openpyxl	>=3.0.0	Yes	Windows_Experiment_GUI, evaluate_model, process_run
pyserial	>=3.5	Yes	Both GUIs (serial comms to Portenta)
PyYAML	>=5.4.0	Yes	config_loader
pycocotools	>=2.0.0	Yes	Training, evaluation, synthetic data generators
PySide6	not pinned	NO	GUI_Clean.py
pyqtgraph	not pinned	NO	GUI_Clean.py (live pressure chart)
torch / PyTorch	not pinned	NO	All Detectron2 scripts, process_run
detectron2	not pinned	NO	Training, inference, evaluation, process_run
scipy	not pinned	NO	Cone_4.py (curve_fit for sigmoid fitting)
tqdm	not pinned	NO	train_detectron2.py (progress bars)

Optional / Conditional Python Packages

Package	Listed in requirements.txt?	Used By	Notes
pyphantom	Commented out (>=3.11.11.806)	Both GUIs	Phantom camera SDK — vendor install
pyvisa	Commented out (>=1.11.0)	Both GUIs	Tektronix AFG1062 function generator
win32api (pywin32)	No	config_loader	Windows-only volume label detection

Arduino Libraries

Library	Used By	Notes
TMCStepper	Pressure_Motor_Portenta.cpp	TMC5160 stepper driver communication (SPI)
AccelStepper	Pressure_Motor_Portenta.cpp	Non-blocking stepper motor control

System-Level Dependencies

Dependency	Required By	Notes
CUDA / cuDNN	All Detectron2 scripts	GPU training/inference. Falls back to CPU but impractical for training.
NI-VISA or Tektronix VISA drivers	Windows_Experiment_GUI, GUI_Clean	Required by pyvisa for AFG1062 instrument control
Phantom SDK runtime	Both GUIs	Proprietary Vision Research SDK for Phantom camera (100.100.100.1, 1000fps, 640x480)
Arduino IDE / Portenta H7 board support	Pressure_Motor_Portenta.cpp	For firmware compilation and upload
SPI hardware	Portenta H7 firmware	TMC5160 communicates via SPI (software bit-bang on pins 6/8/9/10)

Hardware Dependencies

Hardware	Controlled By	Protocol
Portenta H7 (Arduino)	Both GUIs	USB Serial (9600 baud)
TMC5160 stepper driver + motor	Firmware	SPI via TMCStepper lib
AliCat pressure controller	Firmware	0-5V analog (PWM out via LLC, ADC in via LLC)
Front/back optical endstops	Firmware	Digital input with pullup (pins 1, 5)
Phantom high-speed camera	Both GUIs	Ethernet (100.100.100.1) via pyphantom SDK
USB webcam	capture_still_image, webcam_preview	USB via OpenCV VideoCapture
Tektronix AFG1062	Both GUIs	SCPI via PyVISA
Raspberry Pi (camera bridge)	Firmware (Serial1 UART)	UART at 115200 baud, CAM: command prefix

C. Notable Issues & Observations

Duplicates

- inference_detectron2.py (root) is a duplicate of AI/Training_Analysis/inference_detectron2.py with a different MODEL_PATH
- setup_training_dataset.py (root) is identical to AI/Training_Analysis/setup_training_dataset.py
- train_detectron2.py (root) is an **evolved** version of AI/Training_Analysis/train_detectron2.py with extra features (CSV logging, visualiser import)
- AI/evaluate_model.py is the canonical evaluation script; AI/Training_Analysis/evaluate_model.py is a near-duplicate missing CSV append

Hardcoded Paths

- Nearly all AI/Training_Analysis/ scripts have Windows-only hardcoded paths (D:\Experiments\...) pointing to specific timestamped training runs
- blur_spray_dataset_generator.py has platform-aware output paths (Windows/Mac/Linux)
- config_loader.py handles cross-platform path resolution with {lacie_drive} placeholders

Undocumented Dependencies (not in requirements.txt)

- PySide6** — required for GUI_Clean.py (the active redesign)
- pyqtgraph** — required for GUI_Clean.py (live pressure chart)
- torch / PyTorch** — required for all ML work
- detectron2** — required for all ML work
- scipy** — required for Cone_4.py
- tqdm** — required for training scripts

Dead Code / Unused

- webcam_preview.py — per project notes, webcam not installed; still in repo
- capture_still_image.py — same situation
- AI/Training_Analysis/test_inference_debug.py — one-time debug script, hardcoded to obsolete training run
- AI/Training_Analysis/verify_installation.py — one-time setup check
- AI/Training_Analysis/visualize_model_predictions.py — hardcoded to obsolete training run

Firmware Notes

- Pressure_Motor_Portenta.cpp has a typo: reveivedDist (should be receivedDist) — used consistently so not a bug, just cosmetic
- Pressure calibration coefficients are empirically derived and hardcoded (offset 0.6, gain 1.032; compensation: (desired + 0.2) / 1.16)
- 3-second periodic handshake (ARDUINO_READY) keeps GUI connection alive
- Camera commands bridged via CAM: prefix over UART1 to Raspberry Pi